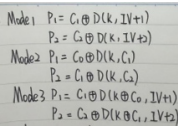


Symmetric Encryption (23 marks): For encrypting the i -th plaintext block P_i , $i = 1, 2, \dots$,
Mode 1: $C_i = P_i \oplus E(K, IV + i)$
Mode 2: $C_i = E(K, C_{i-1} \oplus P_i)$, where $C_0 = IV$.
Mode 3: $C_i = P_i \oplus E(K \oplus C_{i-1}, IV + i)$, where $C_0 = IV$.

- (a) (6 marks)
For all modes write down the equations for *decrypting* block P_1 and P_2 .



- (b) (5 marks)
Which mode is best if a 1-bit error occurs in the transmitted ciphertext. Briefly explain the error propagation for each mode.

Mode 1: $C_i = P_i \oplus E(K, IV + i)$
Decryption: $P_i = C_i \oplus E(K, IV + i)$,
1-bit error in $C_i \rightarrow$ same bit flipped in P_i ; no effect on other bits or blocks.
Propagation: 1 bit in $C_i \rightarrow$ 1 bit error only in P_i .
Mode 2: $C_i = E(K, C_{i-1} \oplus P_i)$, $C_0 = IV$
Decryption: $P_i = D(K, C_i) \oplus C_{i-1}$,
1-bit error in $C_i \rightarrow D(K, C_i)$ $\approx$ random $\rightarrow$ whole P_i corrupted.
For $P_{i+1} = D(K, C_{i+1}) \oplus C_i$; the bad bit in C_i flips one bit in P_{i+1} .
Blocks after $i + 1$ correct.
Propagation: 1 bit in $C_i \rightarrow$ all bits wrong in P_i + 1 bit wrong in P_{i+1} .
Mode 3: $C_i = P_i \oplus E(K \oplus C_{i-1}, IV + i)$, $C_0 = IV$,
Decryption: $P_i = C_i \oplus E(K \oplus C_{i-1}, IV + i)$,
1-bit error in $C_i \rightarrow$ same bit flipped in P_i (keystream uses correct C_{i-1}).
Block $i + 1$ uses $E(K \oplus C_i, IV + i + 1)$; bad $C_i \rightarrow$ wrong keystream $\rightarrow$ all bits of P_{i+1} wrong.
Later blocks unaffected.
Propagation: 1 bit in $C_i \rightarrow$ 1 bit error in P_i + all bits wrong in P_{i+1} .

- (c) (4 marks)
Suppose you have an encrypted file containing 100 ciphertext blocks ($C_1 - C_{100}$). The file stores only the ciphertext and you have none of the intermediate values you generated when you encrypted the file. You wish to decrypt block 51 (C_{51}). How many encrypt/decrypt block cipher operations do you need if the file was encrypted with *Mode 1*, and how many if it was encrypted with *Mode 3*?

We have all ciphertext blocks $C_1, \dots, C_{100}$.

- Mode 1:**
 $P_{51} = C_{51} \oplus E(K, IV + 51) \rightarrow$ 1 encryption.
• Mode 3:
 $P_{51} = C_{51} \oplus E(K \oplus C_{50}, IV + 51) \rightarrow$ need C_{50} (stored) and one call to E .
So also 1 encryption.
(d) (4 marks)
You have the same file setup as in 1.c. You now wish to modify block 51 you previously decrypted and write it back into the file. How many encrypt/decrypt block cipher operations do you need if the file was encrypted with *Mode 1*, and how many if it was encrypted with *Mode 3*?

You already decrypted block 51 once.

- Mode 1:**
New ciphertext $C_{51} = P_{51} \oplus E(K, IV + 51)$.
Other ciphertext blocks unchanged.
 $\rightarrow$ 1 encryption.
• Mode 3:
New $C_{51} = P_{51} \oplus E(K \oplus C_{50}, IV + 51)$ (1 encryption).

- Decrypt 52–100 once to get their plaintexts: 49 decryptions.

- Re-encrypt 51–100 with new chaining: 50 encryptions.
 $\rightarrow$ 99 block-cipher operations in total.

- (e) (4 marks)
You are asked to implement encryption on a very low-resource payment card. There is not enough memory to implement a full block cipher such as AES or DES. The card can only implement a one-way function $f(k, m)$, where K is the secret symmetric key and m is a block of data. Which of the cipher modes given would you use? Explain your answer.

Only operation available: **one-way** $f(K, m)$ with **fixed key** K .

No decryption; key **cannot change per block**.

Mode 2 needs $D(K, \cdot) \rightarrow$ impossible.

Mode 3 needs $E(K \oplus C_{i-1}, \cdot) \rightarrow$ key varies $\rightarrow$ cannot use fixed K .

Mode 1 only needs $E(K, IV + i) \rightarrow$ can be implemented as $f(K, IV + i)$.

2. Public Key Cryptography and Message Integrity (22 marks):

- (a) (3 marks)
We directly sign a message $m=99$ with RSA ($e=7$ $d=43$ $n=77$). Find another message that will give us the same signature. Explain why this is possible.

Reason:
RSA signing computes $s = m^d \bmod n$.
Since $99 \equiv 22 \pmod{77}$, we have
 $99^{43} \equiv 22^{43} \pmod{77}$.
So signing 99 or 22 gives the same signature value.
More generally, any message $m = 99 + k \cdot 77$ (for integer k),

will produce the same signature, because RSA works modulo n .

- (b) (9 marks)
Answer the following questions related to the ElGamal algorithm. ElGamal with public values g and p works as follows:
Private key x is a value between 0 and $p - 1$, and public key $y = g^x \bmod p$.
Ciphertext $C = (A, B)$ where $A = g^r \bmod p$ and $B = M \cdot y^r \bmod p$, with M being the message and r a random value between 0 and $p - 1$.
 C is decrypted by calculating $K = A^x \bmod p$ and $M = B \cdot K^{-1} \bmod p$.
i) If $g = 3$, $p = 19$, $y = 14$ and $C = (2, 14)$ what is M ? Show calculation. (6)

- ii) Name and briefly explain the mathematical problem that ensures the security of ElGamal. What would an attacker gain if it could solve this problem? (3)

Find x from $y = g^x \bmod p$:
 $3^x \equiv 14 \pmod{19}$. Check small exponents $\rightarrow 3^{13} \equiv 14 \pmod{19}$, so $x = 13$.

$K = A^x \bmod p = 2^{13} \bmod 19 = 3$.
 $K^{-1} \equiv 3^{-1} \equiv 13 \pmod{19}$ (since $3 \cdot 13 \equiv 39 \equiv 1$).

$M = B \cdot K^{-1} \bmod 19 = 14 \cdot 13 \bmod 19 = 182 \bmod 19 = 11$.

- (ii)
Security relies on the **Discrete Logarithm Problem (DLP)**:
given $g, p, y = g^x \bmod p$, it is hard to find x .

If an attacker could solve DLP, they could recover the private key x .
Then for any ciphertext (A, B) they compute $K = A^x \bmod p$ and get the plaintext $M = B \cdot K^{-1} \bmod p$.

So breaking DLP $\Rightarrow$ recover all messages $\Rightarrow$ ElGamal confidentiality is broken.

- (c) (4 marks)
For RSA, if $n = 15$, list all the possible public/private key pairs (e, d) . Show calculation.

- Factor n and compute $\varphi(n)$:
 $n = 15 = 3 \cdot 5 \Rightarrow \varphi(n) = (3 - 1)(5 - 1) = 2 \cdot 4 = 8$.
- Valid public exponents e must satisfy $1 < e < \varphi(n)$ and $\gcd(e, \varphi(n)) = 1$.
So $e \in \{3, 5, 7\}$ (and $e = 1$ if you allow the trivial case).
- Find d such that
 $ed \equiv 1 \pmod{\varphi(n)} = \pmod{8}$.

- For $e = 3$; $3 \cdot 3 = 9 \equiv 1 \pmod{8} \Rightarrow d = 3$.

- For $e = 5$; $5 \cdot 5 = 25 \equiv 1 \pmod{8} \Rightarrow d = 5$.

- For $e = 7$; $7 \cdot 7 = 49 \equiv 1 \pmod{8} \Rightarrow d = 7$.

- (Optionally: $e = 1, d = 1$ also satisfies $ed \equiv 1 \pmod{8}$ but is insecure.)

All non-trivial RSA key pairs for $n = 15$:
 $(e, d) \in \{(3, 3), (5, 5), (7, 7)\}$ (and $(1, 1)$ if you include the trivial pair).

- (d) (6 marks)
Assume h is a hash with output length 100 bits and MAC is a Message Authentication Code with output length 128 and key size 64. What is the average effort (number of hash or MAC computations) an attacker needs to do the following? Give your answer in 2^x format and briefly explain your answer.

- i) Find any message x and message y so that $h(y) = h(x)$. (2)
ii) Once the attacker is given message x and $MAC(x)$, find any message x and message y so that $MAC(x) = MAC(y)$. (4)

(d)(i)
Hash output = 100 bits $\rightarrow 2^{100}$ values.
Collision search $\Rightarrow$ birthday bound $\approx 2^{100/2} = 2^{50}$ hash evaluations.
(d)(ii)
MAC tag = 128 bits $\rightarrow 2^{128}$ values.
Collision MAC(x)=MAC(y) $\Rightarrow$ birthday bound $\approx 2^{128/2} = 2^{64}$ MAC evaluations.
(Key size 64 bits is irrelevant for collision cost; it's set by the 128-bit tag.)

- (a) (4 marks)
i) Show how you would use asymmetric cryptographic between only two parties (no trusted 3rd party) to do *key transport*. (2)
ii) Show how you would use asymmetric cryptographic between only two parties (no trusted 3rd party) to do *key agreement*. (2)

(i) Key transport (A $\rightarrow$ B, no TTP, using public-key encryption)

- Alice generates random session key K .
- Alice sends $C = \text{Enc}_{PK_B}(K)$ to Bob.
- Bob computes $K = \text{Dec}_{SK_B}(C)$.
 $\rightarrow$ Asymmetric crypto is used only to transport a symmetric key chosen by Alice.

(ii) Key agreement (A $\leftrightarrow$ B, no TTP, using Diffie–Hellman)

Public parameters: prime p , generator g .

- Alice chooses secret a , sends $A = g^a \bmod p$ to Bob.
- Bob chooses secret b , sends $B = g^b \bmod p$ to Alice.
- Alice computes $K = B^a \bmod p$.
- Bob computes $K = A^b \bmod p$.

Since $B^a = (g^b)^a = g^{ab} \equiv (g^a)^b = A^b \pmod{p}$, both obtain the same shared key $K = g^{ab} \bmod p$.

which is jointly established (neither party chose it alone).

- (b) (6 marks)
Consider the mutual authentication protocol below. R_x is a nonce generated by x .

- $A \rightarrow B : R_A$
- $A \leftarrow B : (R_A + 1, R_B)_{A_{\text{public}}}$
- $A \rightarrow B : R_B$

Discuss the security of this protocol. Your discussion must consider if the security objectives of the protocol is met and explain why or why not. If these are not met then fix the protocol (without adding any additional keys/cryptographic mechanism that Alice and Bob is not already able to perform, i.e. they can only do public key encryption and generate nonces)

Make R_A secret so only B can respond:

- $A \rightarrow B : \{R_A\}_{PK_B}$
- $B \rightarrow A : \{R_A + 1, R_B\}_{PK_A}$
- $A \rightarrow B : R_B$

- Anyone sees R_A and knows PK_A .

- Attacker M can impersonate B :

- $A \rightarrow M(B) : R_A$
- $M(B)$ chooses R_M , sends $(R_A + 1, R_M)_{PK_A}$ to A .
- A decrypts, checks $R_A + 1$, returns R_M .

- B was never involved, but A thinks she talked to B .
 $\rightarrow$ Protocol gives only **“A authenticated to B”**, not mutual authentication.

- (c) (3 marks)
For the protocol in 3.b, B needs the public key of A . How would B check if it has a valid public key for B ?

B must check that the public key really belongs to A and is still valid.

Use a **certificate**: B gets $Cert_A = \{A, PK_A, \text{exp}_1\}_{SK_{CA}}$.

B verifies:
• CA's signature on $Cert_A$ with PK_{CA} ;
• identity field = “A”;
• certificate not expired / not revoked.

If OK, B accepts PK_A from $Cert_A$ as A 's valid public key.

- (d) (10 marks)
Design a protocol that will establish a new shared secret key K between Alice and Bob with the help of a third party. You can only use asymmetric cryptographic mechanisms and timestamps. At the end of the protocol the following conditions must be met:
- Mutual authentication of Alice and Bob; mutual authentication of TTP to Alice and unilateral authentication of TTP to Bob.
 - Alice must have key control.
 - No explicit key authentication required.
- Your answer should be in the form of a protocol diagram between the parties, and you must clearly state your assumptions, explain all actions performed by both parties and define all your variables and notation.

Assumptions / notation (very compact)

- Parties: Alice (A), Bob (B), trusted third party (T).
- Key pairs: (PK_X, SK_X) for $X \in \{A, B, T\}$. Everyone knows PK_X ; T knows PK_A, PK_B .
- $\{M\}_{\{PKX\}}$ = public-key encryption of M with PK_X .
- $[M]_{\{SKX\}}$ = digital signature on M with SK_X .
- T_A, T_T, T_K, T_B = timestamps.
- $Cert_A = \{A, PK_A, \text{exp}\}_{\{SK_T\}}$ (A's certificate signed by T).

Protocol (A, T, B)

- $A \rightarrow T : \{A, B, T_A\}_{\{SK_A\}}$
– A requests to talk to B; T verifies the signature $\rightarrow$ authenticates A, checks T_A (fresh).
- $T \rightarrow A : \{A, B, PK_B, T_A, T_T\}_{\{SK_T\}}$
– T sends B's public key and timestamp; A verifies with $PK_T \rightarrow$ authenticates T, gets trusted PK_B .
 $\rightarrow$ mutual auth $A \leftrightarrow T$.
- A chooses fresh session key K , timestamp T_K .
 $A \rightarrow B : \{A, K, T_K\}_{\{PK_B\}}, \{A, B, K, T_K\}_{\{SK_A\}}, Cert_A$
– B verifies $Cert_A$ with $PK_T \rightarrow$ authenticates T (unilateral $T \rightarrow B$) and obtains PK_A .
– B verifies $\{A, B, K, T_K\}_{\{SK_A\}}$ with $PK_A \rightarrow$ authenticates A and that A chose K for B and it is fresh.
– B decrypts $\{A, K, T_K\}_{\{PK_B\}}$ with SK_B to obtain K .
- $B \rightarrow A : \{A, B, K, T_K, T_B\}_{\{SK_B\}}$
– A verifies with PK_B from step 2, checks $T_B \rightarrow$ authenticates B and that B knows K .

Properties (checked very briefly)

- New shared secret key k established; Alice has key control (she generates k).
- **Mutual auth $A \rightleftharpoons B$:** B authenticates A in step 3 (A's sig); A authenticates B in step 4 (B's sig).
- **Mutual auth $A \rightleftharpoons T$:** T verifies A's sig (1); A verifies T's sig (2).
- **Unilateral auth $T \rightarrow B$:** B only verifies T's signature on Cert_A (3); T never needs to authenticate B.
- Only **asymmetric crypto** (sig/encrypt) and **timestamps** are used; no explicit "key authentication" message beyond what is implied by the signed fields.

(a) (8 marks)

i) You wish to send a data packet between host A and host B. What IPsec mode and protocol would you choose if you need to authenticate the packet (including the IP header) but are not concerned with confidentiality? Explain how a basic packet with fields $[IPHeader|Data]$ would be modified as it is sent from A to B using your choices. (3)

ii) You wish to send a data packet between host A and host D. What IPsec mode and protocol would you choose if you wish to authenticate the packet (excluding the IP header) and provide data confidentiality? Explain how a basic packet with fields $[IPHeader|Data]$ would be modified as it is sent from A to D using your choices. (5)

(i) $A \rightarrow B$, auth incl. IP header, no confidentiality

- Use **IPSec AH in transport mode** (host-to-host).
- Original packet: $[IP \text{ hdr} | Data]$
- After AH (transport): $[IP \text{ hdr} | AH \text{ hdr} | Data]$
- AH adds an Authentication Header between IP header and data.
- AH integrity-protects (most of) the IP header + the payload, but does not encrypt anything.

(ii) $A \rightarrow D$, auth excl. IP header, with confidentiality

- Use **IPSec ESP in transport mode** (host-to-host).
- Original packet: $[IP \text{ hdr} | Data]$
- After ESP (transport):
 $[IP \text{ hdr} | ESP \text{ hdr} | \text{Encrypted } \{Data, padding, ESP \text{ trailer}\} | ESP \text{ auth}]$
– ESP encrypts the payload (Data + ESP trailer), and ESP auth (ICV) gives integrity/authentication for the encrypted payload (and ESP header).
- The outer IP header is **not** included in the integrity check, matching "authenticate excluding the IP header".

(b) (4 marks)

You are opening up a new online business selling cupcakes. You would like to make sure that clients can verify that they are actually visiting your website, and you would also like to prevent attackers from seeing what the client is purchasing and payment details. What prominent security protocol would you use? What security services does this protocol provide to achieve the security you want your system to have.

Use TLS/SSL (HTTPS).

Security services it provides (relevant to the question):

- **Server authentication** – browser checks the server's X.509 certificate (signed by a trusted CA) so the client can verify it is really your cupcake site.
- **Confidentiality** – all HTTP data (what is ordered, card details, cookies, etc.) is encrypted on the wire, so eavesdroppers cannot see it.
- **Integrity / tamper detection** – MAC/AEAD in TLS ensures packets cannot be modified without being detected, so attackers cannot change orders or payment data in transit.

(c) (4 marks)

- What is a DoS attack and which security services it is linked to. (2)
- Explain briefly the difference between DDoS and DoS attacks. (2)

If a DoS (Denial of Service) attack is an attack that makes a system or service unavailable to legitimate users (by exhausting CPU, bandwidth, connections, etc.).

DoS (拒绝服务) 攻击是指系统或服务对合法用户不可用的状态 (耗尽 CPU、带宽、连接等资源)。

It is mainly linked to the **availability** security service (sometimes indirectly impacts integrity if data is lost during outage).

它主要影响的是安全目标的可用性 (有时在攻击导致数据丢失时也会间接影响完整性)。

If a DoS attack usually comes from a **single** attacking host or a small number of sources.

普通 DoS 通常来自单个攻击主机或少量来源。

A DDoS (Distributed DoS) attack is launched from **many compromised machines** (a botnet) at the same time, making it much harder to block because traffic comes from many different IPs.

DDoS (分布式拒绝服务) 是由大量被控制的机器 (僵尸网络) 同时发起。由于流量来自许多不同 IP，更难以过滤和阻断。

(a) (3 marks)

Give your own example of two-factor authentication and explain what factors are used.

Example: Online banking login using **password + SMS one-time code**.

- Factor 1: password $\rightarrow$ "something you know".
- Factor 2: phone/SMS OTP $\rightarrow$ "something you have".

(b) (5 marks)

It is good practice not to store user's personal information, including passwords, in plaintext. We therefore store the hash of the password instead. Assume that the user's password is a 8 characters (each character can be 0-9, a-z, A-Z).

- If we have a password file with each entry $s, h(pwd, s)$ and s is a 8-bit number how many dictionary entries would an attacker need to cover all possible passwords? (2)
- If we have a password file with a secret salt – entry is only $h(pwd, s)$ – and s is a 8-bit number how many dictionary entries would an attacker need to cover all possible passwords? How much computation effort would we need to verify a signature?(3)

Alphabet size: 0-9, a-z, A-Z $\Rightarrow$ 62 chars.

8-char password $\Rightarrow$ 62^8 possible passwords.

i) Public per-user salt, entry = $\{s, h(pwd, s)\}$, 8-bit s .

Attacker wants precomputed dictionary covering **all passwords and all salts**:

needed entries = $62^8 \times 2^8$.

ii) Secret global salt, entry only $h(pwd, s)$, 8-bit s .

To cover all passwords for all possible secret salts, attacker again needs

$62^8 \times 2^8$ dictionary entries (separate table per possible salt).

Verifying one password candidate once you know the real secret salt still needs only **1 hash computation** on the server.

(c) (3 marks)

You are responsible for making sure a bank provides a secure online website for their customers. Your customers keep getting fooled by phishing attacks and attackers are using their credentials to log in and transfer money to the attacker's account.

How could you change your password log in procedure to be resistant to phishing?

Why does your idea prevent phishing?

(c) Phishing-resistant login

Change from simple "username + static password" to, e.g., **public-key / client-certificate login** or **hardware token-based challenge-response**:

- Bank's website sends a challenge over TLS.
- User's browser or token proves possession of a private key (or secret) without revealing it.
- A fake phishing site cannot learn any reusable secret (private key never leaves device, token output is bound to the real bank's challenge), so stolen "credentials" from the phishing page are useless.

(d) (2 marks)

Look at the segment of pseudo-code below and explain what kind of malware is likely to contain such code.

```
Scan local network;
For each host found do:
{
    Open connection;
    Send copy of malware;}
}
```

(d) Malware type

Code that scans the local network and sends a copy of itself to every host is characteristic of a **network worm (self-propagating worm)**.

(e) (3 marks)

Currently there is a need for a system ensuring that people under quarantine for COVID-19 remain inside their home. Assume there is a system that issues each user with a wrist band, which has a printed QR code on it. The user must scan this QR code with a mobile every hour and the mobile then uploads the user's ID and current location (from the mobile) to the central system. A person makes a copy of their QR code, leaves their phone at home and gets someone to take photo of QR code copy every hour. Considering what you have learned about user authentication – what security issue does this attack expose? How would one fix this?

(e) COVID QR-code wristband scheme

Issue exposed: the **system authenticates the QR code, not the person** $\Rightarrow$ the QR code is a cloneable, **replayable token** ("what you have" only, and even that can be copied). Attacker can duplicate the code and leave phone at home, so presence/location is faked.

Fix idea: bind authentication to an **unclonable, cryptographic token** or **to the user themselves**, e.g.:

- Wristband contains a chip with a secret key; phone sends a fresh challenge, band returns a MAC/signature; a mere photo of the QR cannot answer the challenge.
- Optionally combine with an additional factor (biometric or PIN on the phone) so that both "who you are" / "what you have" are checked.